CPCS-324 GROUP PROJECT PART II

King Abdulaziz University
Faculty of Computing and Information Technology
Department of Computer Science
First Semester 2026

Section: BAR, DBR – **Project ID:** Group 22

Student Name	Student ID	Section
Leader: Arwa Ali AL-Qahtani	2306986	BAR
Rahaf Rajab AL-Zahrani	2307168	BAR
Jood Faisal AL-Zahrani	2307623	BAR
Hanan Al-Mehmadi	2308402	DBR

Table of Contents

List of Figures 2
List of Tables 3
1. Introduction 4
1.1 Project Overview 4
1.2 Objectives and Scope of Part II 4
2. Algorithms and Methods 6
2.1 Huffman Coding (Greedy Variable-Length Encoding) 6
2.1.1 Frequency Calculation 6
2.1.2 Huffman Tree Construction (Greedy with PriorityQueue) 6
2.1.3 Code Generation (Prefix-Free Codes) 7
2.1.4 Encoding and Decoding 8
2.2 Fixed-Length (8-bit) Encoding 10
2.2.1 Code Assignment 10
2.2.2 Encoding 11
2.2.3 Decoding 11
2.3 Supporting Classes and Overall Flow 12
2.4 Data Structures Used (Priority Queue, Tree, Maps) 13
3. Implementation Overview 13
3.1 Random Text File Generation (50 KB, 200 KB, 1 MB) 13
3.2 Encoder and Decoder Design for Both Methods 14
3.3 Correctness Verification (Comparing Original vs Decoded Text) 15
4. Experimental Setup 16
4.1 Test Files and Input Description 16
4.2 Evaluation Metrics: Size, Compressed Size, Compression Ratio, Time 16
4.3 Hardware and Software Environment 17
4.4 Experiment Procedure 17
5. Results 18
6. Analysis and Visualization 19
6.1 File Size vs Compression Ratio 19
6.2 File Size vs Running Time 19
6.3 Compression Ratio vs Running Time 20
6.4 Discussion of Compression Efficiency and Trade-Offs 20
7. Conclusion 21
7.1 Summary of Findings 21
7.2 Practical Implications and Possible Future Extensions 22

8. References 22
9. Appendix 23
A. Sample Huffman Code Table (Character → Codeword) 23
B. Sample Console Outputs and Screenshots (including verification) 26
C. Key Source Code Snippets (Main Functions / Classes) 26

List of Figures

Figure 1: calculateFrequency Method in HuffmanCoding Class 6
Figure 2: buildTree Method in HuffmanCoding Class 6
Figure 3: Creating a Leaf Node for Each (character, frequency) Pair in freqMap 7
Figure 4: Making the Remaining Node the Root 7
Figure 5: generateCodes Method in HuffmanCoding Class 7
Figure 6: huffmanCodes is Responsible for Storing the Generated Codes When Leaf Nodes are Reached 8
Figure 7: The Retrieval of The Complete Mapping from Characters to Codewords 8
Figure 8: Implementation of Encoding 8
Figure 9: Implementation of Decoding 9
Figure 10: Validate Correctness of Decoded Text in NewHuffmanCoding Class 9
Figure 11: generateCodes Method in FixedLengthCoding Class 10
Figure 12: Storing the Mapping in fixedCodes 10
Figure 13: encode Method in FixedLengthCoding Class 11
Figure 14: decode Method in FixedLengthCoding Class 11
Figure 15: Ensuring the Correctness of the Fixed-length Encoder/Decoder in NewHuffmanCoding.main 12
Figure 16: generateRandomFile Method in RandomTextFileGenerator Class to Generate Random Text Files 12
Figure 17. Compression ratio of Huffman and Baseline encoding across different file sizes. 19
Figure 18. Total running time (encoding + decoding) for Huffman and Baseline methods at different file sizes. 19
Figure 19. Association between compression ratio and total running time for both encoding methods 20
Figure 20. Huffman Code Table generated for the 50KB input file (file_50KB.txt) 23

Figure 21. Huffman Code Table generated for the 200KB input file (file_200KB.txt).....24
Figure 22. Huffman Code Table generated for the 1MB input file (file_1MB.txt) 25
Figure 23. Performance Comparison Between Huffman Coding and Fixed-Length Encoding 26
Figure 24. HuffmanNode Class Implementation 26
Figure 25. Core HuffmanCoding Methods for Building the Tree and Generating Codes.....27
Figure 26. Core HuffmanCoding Methods for Building the Encoding and Decoding.....28
Figure 27. Huffman Encoding Workflow in NewHuffmanCoding Class.....29
Figure 28. Fixed-Length Encoding Workflow for Performance Comparison 29
Figure 29. PerformanceRecord Class for Storing Encoding and Compression Metrics.....30
Figure 30. Fixed-Length Encoding and Decoding Implementation 30
Figure 31. RandomTextFileGenerator Class for Creating Synthetic Test Files.....31
Figure 32. FileUtils Class for File Input and Output Operations.....31

List of Tables

Table 1. Summary of original size, compressed size, compression ratio, and runtime measurements for Huffman and Baseline methods.....18

1. Introduction

1.1 Project Overview

Data compression is a fundamental problem in computer science [1], especially when transmitting large files over a network or storing them efficiently and correctly [2] [3]. In this project (Part II of the CPCS 324 group project), we implement and compare two text encoding methods:

- **Huffman coding:** a greedy variable-length encoding algorithm that builds an optimal prefix-free binary tree based on character frequencies [4].
- **Fixed-length encoding:** a simple baseline method that assigns a fixed 8-bit code to every character.

The system generates three random text files consisting only of uppercase letters A-Z with approximate sizes of 50 KB, 200 KB, and 1 MB. For each file, we apply both encoding methods, compress the text, decode it back to the original, and measure several performance metrics such as file sizes, compression ratio, and encoding/decoding running time.

This work demonstrates how greedy strategies can be used beyond graph problems (as in Part I with MST) and applied to real-world scenarios such as data compression and network transmission.

1.2 Objectives and Scope of Part II

The main objective of Part II is to design, implement, and experimentally evaluate Huffman coding as a greedy algorithm and compare it with a fixed-length baseline. More specifically, this part aims to:

- Implement Huffman codes that:
    1. Computes character frequencies.
    2. Builds a Huffman tree using a priority queue.
    3. Generates prefix-free variable-length bit codes.
    4. Encodes and decodes text correctly.
- Implement a fixed-length 8-bit encoding scheme that:
    1. Assigns a unique 8-bit binary code to each character used in the file.
    2. Encodes and decodes text using only fixed-size codewords.
- Build encoders and decoders for both methods and verify correctness by comparing decoded text with the original text.

- Measure, for each file and each method:
    1. Original file size and compressed size (in bytes).
    2. Compression ratio.
    3. Encoding and decoding running time (in milliseconds).
- Analyze and discuss the trade-off between compression efficiency and runtime performance, supported by tables and plots.

The scope is limited to randomly generated text files over the alphabet A–Z, stored as plain .txt files. Other compression algorithms or real-world file formats are outside the scope of this project.

2. Algorithms and Methods

This section describes how we implemented the two encoding methods using our Java classes in the g22_Code package.

2.1 Huffman Coding (Greedy Variable-Length Encoding)

The HuffmanCoding class implements Huffman encoding and decoding. The algorithm follows the standard greedy approach:

2.1.1 Frequency Calculation

We first compute the frequency of each character in the input text using the method in:

```java
// Counts frequency of each character in the text
public Map<Character, Integer> calculateFrequency(String text) {
    Map<Character, Integer> freqMap = new HashMap<>();
    for (char c : text.toCharArray()) {
        freqMap.put(c, freqMap.getOrDefault(c, 0) + 1);
    }
    return freqMap;
}
```

Figure 1: calculateFrequency Method in HuffmanCoding Class

This method iterates over the text and stores counts in a HashMap<Character, Integer>. Each character (A–Z) becomes a key, and its frequency in the file is the value. This frequency map is used as the input to build the Huffman tree.

2.1.2 Huffman Tree Construction (Greedy with PriorityQueue)

The method:

```java
// Builds Huffman Tree using a PriorityQueue (Greedy Approach)
public void buildTree(Map<Character, Integer> freqMap) {
    PriorityQueue<HuffmanNode> pq = new PriorityQueue<>();

    for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {

        pq.add(new HuffmanNode(entry.getKey(), entry.getValue()));
    }

    // Merge nodes until a single root remains
    while (pq.size() > 1) {
        HuffmanNode left = pq.poll();
        HuffmanNode right = pq.poll();
        HuffmanNode newNode = new HuffmanNode(left.freq + right.freq, left, right);
        pq.add(newNode);
    }

    root = pq.poll();
    generateCodes(root, "");
}
```

Figure 2: buildTree Method in HuffmanCoding Class

constructs the Huffman tree using a min-priority queue of HuffmanNode objects:

- For each (character, frequency) pair in freqMap, we create a leaf node and insert it into a PriorityQueue<HuffmanNode>:

```java
for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {

    pq.add(new HuffmanNode(entry.getKey(), entry.getValue()));
}
```

Figure 3: Creating a Leaf Node for Each (character, frequency) Pair in freqMap

- The HuffmanNode class implements Comparable<HuffmanNode>, where compareTo compares nodes by their freq value. This ensures that the priority queue always returns the node with the smallest frequency first.
- While the priority queue has more than one node, we:
  1. Remove the two nodes with the smallest frequencies (left and right).
  2. Create a new internal node with:
     - `freq = left.freq + right.freq`
     - left and right as its children.
  3. Insert this new node back into the priority queue.
- When only one node remains, it becomes the root of the Huffman tree:

```java
root = pq.poll();
```

Figure 4: Making the Remaining Node the Root

This process is greedy because, at each step, it always merges the two least-frequent nodes, which is proven to produce an optimal prefix-free code for the given frequency distribution.

2.1.3 Code Generation (Prefix-Free Codes)

Once the tree is built, the method:

```java
// Recursively generate prefix-free Huffman codes
private void generateCodes(HuffmanNode node, String code) {
    if (node == null) return;

    if (node.isLeaf()) {
        huffmanCodes.put(node.ch, code);
    }

    generateCodes(node.left, code + "0");
    generateCodes(node.right, code + "1");
}
```

Figure 5: generateCodes Method in HuffmanCoding Class

is used recursively to generate Huffman codes:

- We start from the root with an empty string (“”).
- For each recursive step:
  - Moving to the left child appends “0” to the code.
  - Moving to the right child appends “1” to the code.
- When we reach a leaf node, its character node.ch is assigned the current code string and stored in:

```
private Map<Character, String> huffmanCodes = new HashMap<>(); // Stores generated codes
```

Figure 6: huffmanCodes is Responsible for Storing the Generated Codes When Leaf Nodes are Reached

The complete mapping from characters to codewords can be retrieved via:

```
public Map<Character, String> getCodes() {
    return huffmanCodes;
```

Figure 7: The Retrieval of The Complete Mapping from Characters to Codewords

In NewHuffmanCoding.main, we print this mapping as a character, Huffman code table for each file, as required.

2.1.4 Encoding and Decoding

- Encoding is implemented by:

```
// Encode the input text using generated Huffman codes
public String encode(String text) {
    StringBuilder encoded = new StringBuilder();
    for (char c : text.toCharArray()) {
        encoded.append(huffmanCodes.get(c));
    }
    return encoded.toString();
}
```

Figure 8: Implementation of Encoding

For each character in the input text, we look up its Huffman code in huffmanCodes and append it to a StringBuilder. The result is a string of '0' and '1' characters representing the compressed bitstream.

- Decoding is implemented by:

```java
// Decode encoded bit string back into original text
public String decode(String encodedText) {
    StringBuilder decoded = new StringBuilder();
    HuffmanNode current = root;

    for (char bit : encodedText.toCharArray()) {
        current = (bit == '0') ? current.left : current.right;

        if (current.isLeaf()) {
            decoded.append(current.ch);
            current = root;
        }
    }
    return decoded.toString();
}
```

Figure 9: Implementation of Decoding

We traverse the Huffman tree according to each bit:

- For bit '0': move to current.left.
- For bit '1': move to current.right.
- When a leaf node is reached, we append the stored character current.ch to the decoded output and reset current back to the root.

This continues until all bits are processed. In NewHuffmanCoding, we validate correctness by checking:

```java
// Ensure decoded text is correct
if (!decoded.equals(text)) {
    System.out.println("Huffman Decoding Error!");
}
```

Figure 10: Validate Correctness of Decoded Text in NewHuffmanCoding Class

ensuring that the Huffman decoder reconstructs the original file contents exactly.

2.2 Fixed-Length (8-bit) Encoding

The FixedLengthCoding class implements the baseline 8-bit fixed-length encoding for comparison with Huffman coding.

2.2.1 Code Assignment

The method:

```java
// Assigns 8-bit binary representation for each character
public void generateCodes(Set<Character> characters) {
    int bits = 8;
    int index = 0;

    for (char c : characters) {
        String binary = String.format("%" + bits + "s", Integer.toBinaryString(index))
                .replace(' ', '0');
        fixedCodes.put(c, binary);
        index++;
    }
}
```

Figure 11: generateCodes Method in FixedLengthCoding Class

creates a unique 8-bit binary code for each character present in the file:

- We maintain an integer index starting from 0.
- For each character c in the set:
    - Convert index to binary using Integer.toBinaryString(index).
    - Left-pad the binary string with zeros to ensure a fixed length of 8 bits using String.format("%8s", ...).
    - Store the mapping in:

```java
private Map<Character, String> fixedCodes = new HashMap<>();
```

Figure 12: Storing the Mapping in fixedCodes

    - Increment index for the next character.

This guarantees that all characters of the file have a distinct fixed 8-bit code, even though the underlying alphabet (A–Z) would theoretically fit in 5 bits.

2.2.2 Encoding

The method:

```java
    // Encode entire text using predefined 8-bit codes
    public String encode(String text) {
        StringBuilder encoded = new StringBuilder();
        for (char c : text.toCharArray()) {
            encoded.append(fixedCodes.get(c));
        }
        return encoded.toString();
    }
```

Figure 13: encode Method in FixedLengthCoding Class

encodes the text as follows:

- For each character c in the input text:
    - Look up its 8-bit code in fixedCodes.
    - Append it to a StringBuilder.

The result is a bit string where every character is represented by exactly 8 bits. This method does not compress the data relative to a standard 8-bit-per-character representation; it is used only as a controlled baseline.

2.2.3 Decoding

The corresponding decoding method is:

```java
    // Decode bit string back into original text
    public String decode(String encodedText, int length) {
        StringBuilder decoded = new StringBuilder();

        for (int i = 0; i < encodedText.length(); i += 8) {
            String code = encodedText.substring(i, i + 8);
            for (char key : fixedCodes.keySet()) {
                if (fixedCodes.get(key).equals(code)) {
                    decoded.append(key);
                    break;
                }
            }
        }
        return decoded.toString();
    }
```

Figure 14: decode Method in FixedLengthCoding Class

- The encoded bitstring is processed in blocks of 8 bits:
    - For each block of 8 bits, we search through fixedCodes to find which character has that code.
    - When a match is found, we append that character to the decoded text.

This process reconstructs the original text, character by character. In NewHuffmanCoding.main, we check correctness with:

```
if (!fixedDecoded.equals(text)) {
    System.out.println("Fixed-Length Decoding Error!");
}
```

Figure 15: Ensuring the Correctness of the Fixed-length Encoder/Decoder in NewHuffmanCoding.main

ensuring that the fixed-length encoder/decoder is also correct.

2.3 Supporting Classes and Overall Flow

Several helper classes support both algorithms:

- **HuffmanNode:** Represents a node in the Huffman tree. Each node stores:
    - char ch: the character (for leaf nodes).
    - int freq: the frequency of that character or the sum of child frequencies.
    - HuffmanNode left, right: child references.

      Implements Comparable<HuffmanNode> so that nodes can be ordered by frequency inside a PriorityQueue.
- **FileUtils:** Provides utility methods:
    - saveFile(String fileName, String content): saves a string to a file.
    - readFile(String fileName): reads the entire file into a string.
    - getFileSize(String fileName) : returns file size in bytes (used later in analysis).
- **RandomTextFileGenerator:** Generates the random test files used as input:

```
public static void generateRandomFile(String fileName, int sizeKB) throws Exception {
    String characters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    Random random = new Random();
    int length = sizeKB * 1024; // Convert KB to bytes

    StringBuilder sb = new StringBuilder(length);

    // Add random characters until file reaches the desired size
    for (int i = 0; i < length; i++) {
        sb.append(characters.charAt(random.nextInt(characters.length())));
    }

    FileUtils.saveFile(fileName, sb.toString());
}
```

Figure 16: generateRandomFile Method in RandomTextFileGenerator Class to Generate Random Text Files

It builds a string of length sizeKB * 1024 using random characters from "ABCDEFGHIJKLMNOPQRSTUVWXYZ" and writes it to disk through FileUtils.

- **PerformanceRecord:** Stores performance measurements for each (file, method) pair:
  - fileName, method, originalSize, compressedSize, compressionRatio, encodeTime, decodeTime.

    These records are later printed and saved for analysis.
- **NewHuffmanCoding (main):** Coordinates the entire process:
  - Generates the three input files.
  - For each file:
    1. Reads the text.
    2. Runs Huffman coding (frequency, tree, codes, encode/decode).
    3. Prints the Huffman character/code table.
    4. Runs fixed-length encoding using the same set of characters.
    5. Measures encoding and decoding times for both methods.
    6. Stores all results in PerformanceRecord objects.
  - Finally, prints a formatted performance table to the console and saves it to "performance_results.txt".

Together, these classes fully implement the requirements for the project: Huffman coding with variable-length encoding, a baseline fixed-length 8-bit encoding, and decoders for both methods that successfully reconstruct the original text.

2.4 Data Structures Used (Priority Queue, Tree, Maps)

In our implementation, we use a HashMap<Character, Integer> to store character frequencies, a PriorityQueue<HuffmanNode> (min-heap) to build the Huffman tree greedily, and a Map<Character, String> to store the final Huffman codewords. The Huffman tree itself is represented using the HuffmanNode class with left and right references. For the fixed-length method, we only use a Map<Character, String> to store the assigned 8-bit codes.

3. Implementation Overview

3.1 Random Text File Generation (50 KB, 200 KB, 1 MB)

In this step, the task is to generate random text files of three different sizes: 50 KB, 200 KB, and 1 MB. Each file will contain random uppercase letters (from A-Z). The files will be saved

in .txt format. The text will be generated using a random selection of characters from the set A-Z.

To achieve this, a random text generation program will be written. For instance, for a 50 KB file, approximately 50,000 characters will be generated, since each character typically occupies one byte. Similarly, for the 200 KB and 1 MB files, 200,000 and 1,000,000 characters will be generated, respectively.

The following requirements need to be met:

• The files must be random in content (i.e., no specific patterns should exist in the text).

• The file sizes must be as close as possible to the target sizes (50 KB, 200 KB, 1 MB).

• The files must be saved in .txt format.

3.2 Encoder and Decoder Design for Both Methods

. Huffman Coding (Variable-Length Encoding):

• Step 1: Calculate the frequency of each character in the file. This will give insight into how often each letter appears.

• Step 2: Store the characters and their frequencies in a priority queue (or min-heap) sorted by frequency in ascending order. The priority queue helps to efficiently retrieve the least frequent characters.

• Step 3: Build the Huffman tree. This is done by repeatedly merging the two nodes (characters) with the smallest frequencies until a single tree remains. Each merge forms a new internal node, and this process continues until all characters are represented in the tree.

• Step 4: Generate the prefix-free binary codewords for each character based on the tree. The binary codewords are determined by traversing the tree from the root: left branches are assigned “0” and right branches are assigned “1”.

• Step 5: Display a table showing each character and its corresponding Huffman codeword. For example:

Char	Huifman Codeword
A	000
B	001
C	01

2. Fixed-Length Encoding: • In this method, every character is assigned a fixed 8-bit code, regardless of its frequency. • The code for each character will be the same length (8 bits), and there will be no optimization based on the character frequencies. • This approach is simpler and doesn't compress the file as efficiently as Huffman coding.
3. Decoder Design: • For Huffman Encoding: A decoder will be implemented that reads the Huffman encoded bitstream and reconstructs the original text. It will navigate through the Huffman tree by following the "0" and "1" values in the encoded stream until it reaches a leaf node, which corresponds to a character. • For Fixed-Length Encoding: A simpler decoder will be used. It reads 8-bit chunks and directly maps them to the corresponding characters in the text. Correctness Verification: After encoding and decoding the text, we will compare the decoded text with the original text. If they match, the encoding and decoding process is correct. If any discrepancies are found, the encoder and decoder will need to be debugged.

3.3 Correctness Verification (Comparing Original vs Decoded Text)

Once the encoding and decoding processes are complete, the decoded text will be compared to the original text to ensure that no errors occurred during encoding or decoding. The process will be verified by checking if the decoded text exactly matches the original text. If any mismatch occurs, it will indicate a potential issue with the encoder or decoder, and debugging will be required.

For both encoding methods (Huffman and Fixed-Length), the following steps will be followed:

• Compare the original text file with the decoded text file.

• Ensure that all characters are accurately restored, with no data loss.

• If the decoded text matches the original text, the encoding/decoding process is deemed correct.

4. Experimental Setup

4.1 Test Files and Input Description

The experimental setup involves using three randomly generated text files of different sizes: 50 KB, 200 KB, and 1 MB. These files will be generated randomly using uppercase letters (A-Z), as described in Requirement 1. The test files will be used to evaluate the performance of both the Huffman coding and Fixed-Length encoding methods. The files should be generated in such a way that they meet the required sizes as closely as possible.

4.2 Evaluation Metrics: Size, Compressed Size, Compression Ratio, Time

For each test file, the following metrics will be collected:

• Original File Size: The size of the file before compression (measured in bytes).

• Compressed File Size: The size of the file after compression using the Huffman and Fixed-Length encoding methods (measured in bytes).

• Compression Ratio: The ratio of the compressed file size to the original file size. This can be calculated as:

$$\text{Compression Ratio} = \frac{\text{Compressed Size (bytes)}}{\text{Original Size (bytes)}}$$

• Encoding Time: The time taken to encode the file using each method (measured in milliseconds).

• Decoding Time: The time taken to decode the file back to its original form (measured in milliseconds).

These metrics will help evaluate the effectiveness of the two encoding methods in terms of file size reduction and computational efficiency.

4.3 Hardware and Software Environment

The experiments will be carried out in the following environment:

• Hardware: [Include details such as the processor type, RAM, etc.]

• Software: The experiments will be conducted using [specify the programming language used, such as Python, Java, etc.], and any relevant libraries (e.g., time library for measuring execution time in Python).

This setup ensures the experiments are conducted under consistent conditions, making the results reproducible.

4.4 Experiment Procedure

1. File Generation: Random text files of 50 KB, 200 KB, and 1 MB will be generated using the random text generator code.
2. Encoding: The files will be encoded using both Huffman coding and Fixed-Length encoding methods.
3. Time Measurement: The time taken for encoding and decoding will be measured using timing functions available in the programming language. For example, System.currentTimeMillis() in Java or time.time() in Python.
4. File Size Measurement: The size of both the original and compressed files will be recorded using file size functions.

5. Compression Ratio Calculation: The compression ratio will be calculated using the formula above.
6. Data Collection: The metrics (original size, compressed size, compression ratio, encoding time, decoding time) will be recorded in a structured format (e.g., tables or spreadsheets) for later analysis and comparison.

The procedure will ensure that accurate and consistent results are obtained for each of the test files.

5. Results

File	Method	Original Size (bytes)	Compressed Size (bytes)	Compression Ratio	Encode Time (ms)	Decode Time (ms)	Total Time (ms)
file_50KB.txt	Huffman	51,200	30,482	0.595	26	6	32
file_50KB.txt	Baseline	51,200	51,200	1.000	0	42	42
file_200KB.txt	Huffman	204,800	122,014	0.596	25	5	30
file_200KB.txt	Baseline	204,800	204,800	1.000	10	70	80
file_1MB.txt	Huffman	1,048,576	624,939	0.596	48	27	75
file_1MB.txt	Baseline	1,048,576	1,048,576	1.000	32	255	287

Table 1. Summary of original size, compressed size, compression ratio, and runtime measurements for Huffman and Baseline methods.

Table 1 provides the numerical results for both encoding methods across all test files. Huffman coding consistently reduces the file size with a compression ratio of about 0.595, while the Baseline method remains at a ratio of 1.0. In terms of performance, Huffman shows stable encoding and decoding times, whereas Baseline decoding becomes slower as the file size increases. These results form the basis for the graphical analysis in Section 6.

6. Analysis and Visualization

6.1 File Size vs Compression Ratio

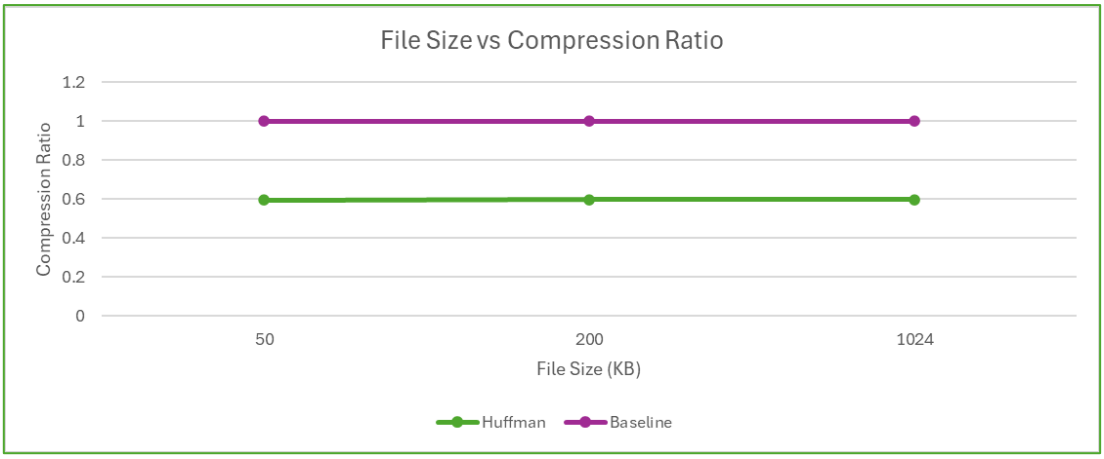

Figure 17. Compression ratio of Huffman and Baseline encoding across different file sizes.

This graph shows that **Huffman** consistently compresses all files to about 59–60% of their original size, regardless of the file size. The **Baseline** method always stays at 1.0, meaning no compression. This confirms that Huffman provides effective size reduction, while Baseline does not reduce file size at all.

6.2 File Size vs Running Time

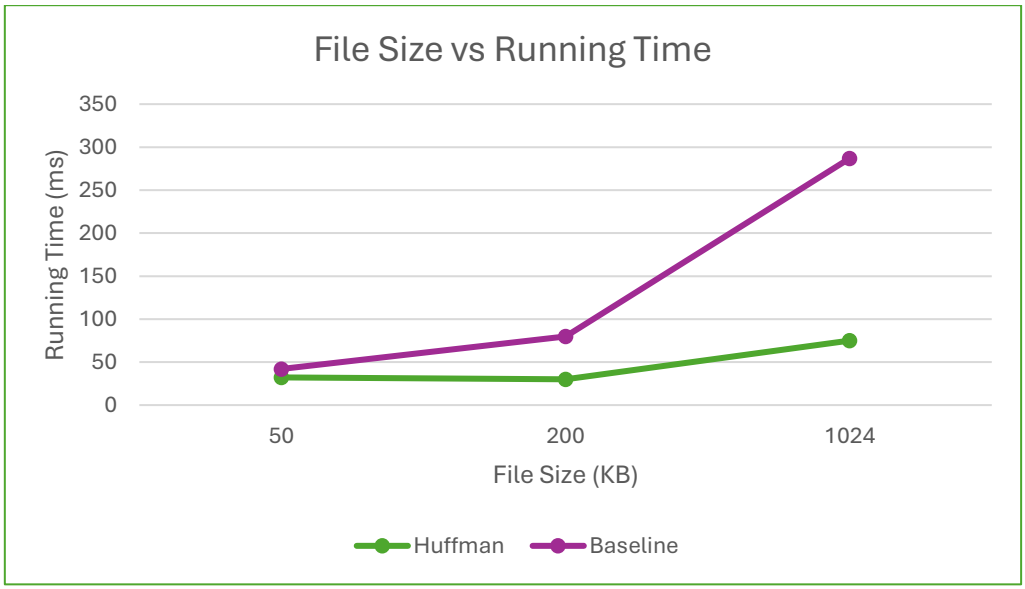

Figure 18. Total running time (encoding + decoding) for Huffman and Baseline methods at different file sizes.

Huffman maintains stable running times as file size increases, whereas the Baseline method shows a sharp rise in total running time, especially for the largest file.

6.3 Compression Ratio vs Running Time

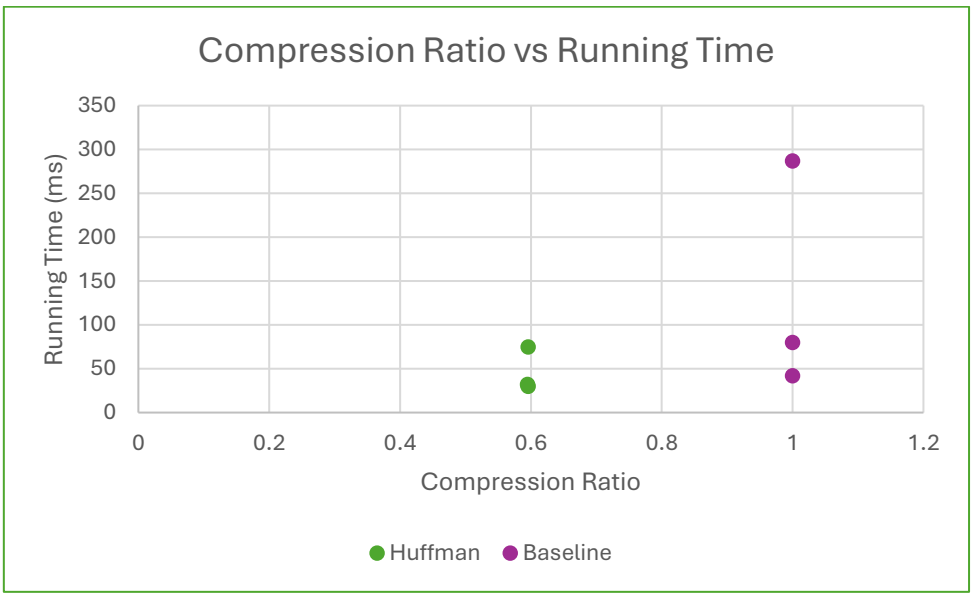

Figure 19. Association between compression ratio and total running time for both encoding methods

Huffman occupies the lower-compression, moderate-time region, while the Baseline method appears at ratio 1.0 with significantly higher running times, forming two distinct performance clusters.

6.4 Discussion of Compression Efficiency and Trade-Offs

The graphical results highlight clear differences between the two encoding methods in terms of both compression quality and runtime performance. Huffman coding consistently achieves a compression ratio of approximately 0.595, reducing file size by around **40%** for all tested files. This demonstrates its effectiveness in assigning shorter codewords to more frequent characters, which explains why its compression ratio remains stable even as file size increases. In contrast, the Baseline method relies on a fixed 8-bit representation for every character, resulting in a constant compression ratio of 1.0 and offering no reduction in size regardless of the input.

From a performance perspective, the plots show that Huffman introduces some overhead during encoding due to the need to compute frequencies, build the priority queue, and construct the Huffman tree. However, it benefits from fast and predictable decoding, as each bit directly guides the traversal of the prefix tree. On the other hand, the Baseline method performs very quick encoding, but its decoding time increases significantly as file size grows because each 8-bit block must be matched and looked up individually. This effect becomes especially noticeable for the 1 MB file, where the Baseline method shows a substantial rise in total running time compared to Huffman.

Overall, the trade-off is clear: Huffman provides meaningful compression while maintaining efficient overall runtime, making it a strong choice when storage space or transmission bandwidth is limited. The Baseline method prioritizes simplicity and may be suitable for small inputs, but it does not scale well for large files and does not offer any compression benefit.

Therefore, Huffman is the more practical and effective encoding technique for real-world scenarios that require both compression efficiency and good performance.

7. Conclusion

7.1 Summary of Findings

This project compared Huffman coding with a Baseline fixed-length encoding method in terms of compression quality and performance. The results showed that Huffman coding consistently achieved a compression ratio of approximately 0.595, reducing file sizes by about 40% across all tested file sizes. In contrast, the Baseline method maintained a compression ratio of 1.0, offering no compression since it uses a constant 8-bit code for every character.

In terms of runtime performance, Huffman required slightly more processing during encoding due to frequency analysis and tree construction, but it demonstrated fast and stable decoding regardless of file size. The Baseline method encoded quickly but experienced a significant increase in decoding time as the input size grew, resulting in much higher total running time for the 1 MB file. Overall, Huffman proved more efficient in both compression effectiveness and overall runtime behavior.

7.2 Practical Implications and Possible Future Extensions

Based on the results, Huffman coding is a useful method when reducing file size or saving storage space is important. It can help make data transmission faster and more efficient because it reduces the amount of data that needs to be sent. Huffman is also a good choice for systems with limited memory, since it compresses text effectively while still keeping decoding fast.

For future work, this project could be extended by testing other compression methods, such as adaptive Huffman, arithmetic coding, or dictionary-based algorithms like LZW, to compare their performance. It would also be helpful to try the algorithms on different types of data, not just random text, to see how they perform in more realistic situations. Another possible extension is to build a simple user interface or tool that allows users to compress and decompress files and see the results directly

8. References

[1] computersciencecafe, "DATA REPRESENTATION 1.7 DATA COMPRESSION," computersciencecafe, [Online]. Available: https://www.computersciencecafe.com/17-data-compression.html. [Accessed 1 December 2025].

[2] S. K. Mitra, "Introduction to Data Compression," geeksforgeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/introduction-to-data-compression/. [Accessed 1 December 2025].

[3] "Data Compression," barracuda, [Online]. Available: https://www.barracuda.com/support/glossary/data-compression.

[4] F. o. d. r. -. H. c. a. H. trees, bbc, [Online]. Available: https://www.bbc.co.uk/bitesize/guides/zd88jty/revision/10.

9. Appendix

A. Sample Huffman Code Table (Character → Codeword)

Output - g22_Code (run)

```
Huffman codes for file: file_50KB.txt
Char     Code
A        0101
B        11000
C        11111
D        0011
E        01111
F        10100
G        0100
H        11011
I        10001
J        01100
K        10011
L        11101
M        10101
N        0010
O        10111
P        0000
Q        01110
R        10110
S        11110
T        11100
U        10010
V        10000
W        11001
X        01101
Y        11010
Z        0001
```

Figure 20. Huffman Code Table generated for the 50KB input file (file_50KB.txt)

Output - g22_Code (run)

```
Huffman codes for file: file_200KB.txt
Char     Code
A        11001
B        01101
C        11011
D        11100
E        0101
F        10100
G        10011
H        10111
I        0011
J        10101
K        0000
L        11111
M        11000
N        11110
O        01110
P        10001
Q        01100
R        0010
S        10000
T        10010
U        01111
V        0001
W        0100
X        11101
Y        10110
Z        11010
```

Figure 21. Huffman Code Table generated for the 200KB input file (file_200KB.txt)

Output - g22_Code (run)

```
Huffman codes for file: file_1MB.txt
Char     Code
A        01101
B        11111
C        0011
D        10011
E        10100
F        11000
G        11110
H        11010
I        0000
J        0010
K        10001
L        10010
M        10111
N        11001
O        11011
P        11100
Q        11101
R        10110
S        10101
T        01100
U        01111
V        0101
W        01110
X        10000
Y        0100
Z        0001
```

Figure 22. Huffman Code Table generated for the 1MB input file (file_1MB.txt)

B. Sample Console Outputs and Screenshots (including verification)

```
File            Method          Original Size        Compressed Size      Compression Ratio    Encode (ms)     Decode (ms)
-----------------------------------------------------------------------------------------------------------------------------
file_50KB.txt   Huffman         51200                30474                0.595                43              20
file_50KB.txt   Fixed-Length    51200                51200                1.000                7               59
file_200KB.txt  Huffman         204800               122019               0.596                34              19
file_200KB.txt  Fixed-Length    204800               204800               1.000                17              99
file_1MB.txt    Huffman         1048576              624986               0.596                81              77
file_1MB.txt    Fixed-Length    1048576              1048576              1.000                43              397
-----------------------------------------------------------------------------------------------------------------------------

Results Saved to: performance_results.txt
BUILD SUCCESSFUL (total time: 2 seconds)
```

Figure 23. Performance Comparison Between Huffman Coding and Fixed-Length Encoding

C. Key Source Code Snippets (Main Functions / Classes)

- HuffmanNode Class:

```java
// Represents a node in the Huffman tree
public class HuffmanNode implements Comparable<HuffmanNode> {
    char ch;                // Character stored in the node
    int freq;               // Frequency of the character
    HuffmanNode left;       // Left child (represents bit '0')
    HuffmanNode right;      // Right child (represents bit '1')

    // Constructor for leaf nodes
    public HuffmanNode(char ch, int freq) {
        this.ch = ch;
        this.freq = freq;
        this.left = null;
        this.right = null;
    }

    // Constructor for internal nodes (merge two nodes)
    public HuffmanNode(int freq, HuffmanNode left, HuffmanNode right) {
        this.ch = '\0';
        this.freq = freq;
        this.left = left;
        this.right = right;
    }

    // Sorting based on frequency for PriorityQueue
    @Override
    public int compareTo(HuffmanNode other) {
        return this.freq - other.freq;
    }

    // Check if node is a leaf (represents a character)
    public boolean isLeaf() {
        return left == null && right == null;
    }
}
```

Figure 24. HuffmanNode Class Implementation

- HuffmanCoding Class:

```java
    // Builds Huffman Tree using a PriorityQueue (Greedy Approach)
    public void buildTree(Map<Character, Integer> freqMap) {
        PriorityQueue<HuffmanNode> pq = new PriorityQueue<>();

        for (Map.Entry<Character, Integer> entry : freqMap.entrySet()) {

            pq.add(new HuffmanNode(ch: entry.getKey(), freq: entry.getValue()));
        }

        // Merge nodes until a single root remains
        while (pq.size() > 1) {
            HuffmanNode left = pq.poll();
            HuffmanNode right = pq.poll();
            HuffmanNode newNode = new HuffmanNode(left.freq + right.freq, left, right);
            pq.add(e: newNode);
        }

        root = pq.poll();
        generateCodes(node: root, code: "");
    }

    // Recursively generate prefix-free Huffman codes
    private void generateCodes(HuffmanNode node, String code) {
        if (node == null) return;

        if (node.isLeaf()) {
            huffmanCodes.put(key: node.ch, value: code);
        }

        generateCodes(node: node.left, code + "0");
        generateCodes(node: node.right, code + "1");
    }

```

Figure 25. Core HuffmanCoding Methods for Building the Tree and Generating Codes

```java
    // Encode the input text using generated Huffman codes
    public String encode(String text) {
        StringBuilder encoded = new StringBuilder();
        for (char c : text.toCharArray()) {
            encoded.append(str: huffmanCodes.get(key: c));
        }
        return encoded.toString();
    }

    // Decode encoded bit string back into original text
    public String decode(String encodedText) {
        StringBuilder decoded = new StringBuilder();
        HuffmanNode current = root;

        for (char bit : encodedText.toCharArray()) {
            current = (bit == '0') ? current.left : current.right;

            if (current.isLeaf()) {
                decoded.append(c: current.ch);
                current = root;
            }
        }
        return decoded.toString();
    }

    public Map<Character, String> getCodes() {
        return huffmanCodes;
    }
}
```

Figure 26. Core HuffmanCoding Methods for Building the Encoding and Decoding

- NewHuffmanCoding Class:

```java
// ================= Huffman Encoding =================
HuffmanCoding huff = new HuffmanCoding();
long start = System.currentTimeMillis();

Map<Character, Integer> freq = huff.calculateFrequency(text);
huff.buildTree(freqMap: freq);

//Print the code for each Char
System.out.println("\nHuffman codes for file: " + file);
System.out.println(x: "Char\tCode");
for (Map.Entry<Character, String> entry : huff.getCodes().entrySet()) {
System.out.println(entry.getKey() + "\t" + entry.getValue());
}
System.out.println();

String encoded = huff.encode(text);

long encodeTime = System.currentTimeMillis() - start;

start = System.currentTimeMillis();
String decoded = huff.decode(encodedText: encoded);
long decodeTime = System.currentTimeMillis() - start;

// Ensure decoded text is correct
if (!decoded.equals(anObject: text)) {
    System.out.println(x: "Huffman Decoding Error!");
}

results.add(new PerformanceRecord(
        fileName: file, method:"Huffman",
        originalSize: text.length(),
        encoded.length() / 8,
        encodeTime,
        decodeTime
));
```

Figure 27. Huffman Encoding Workflow in NewHuffmanCoding Class

```java
// ================= Fixed-Length Encoding =================
FixedLengthCoding fixed = new FixedLengthCoding();
fixed.generateCodes(characters: freq.keySet());

start = System.currentTimeMillis();
String fixedEncoded = fixed.encode(text);
encodeTime = System.currentTimeMillis() - start;

start = System.currentTimeMillis();
String fixedDecoded = fixed.decode(encodedText: fixedEncoded, length:text.length());
decodeTime = System.currentTimeMillis() - start;

if (!fixedDecoded.equals(anObject: text)) {
    System.out.println(x: "Fixed-Length Decoding Error!");
}

results.add(new PerformanceRecord(
        fileName: file, method:"Fixed-Length",
        originalSize: text.length(),
        fixedEncoded.length() / 8,
        encodeTime,
        decodeTime
));
}
```

Figure 28. Fixed-Length Encoding Workflow for Performance Comparison

- PerformanceRecord Class:

```java
// Stores performance measurement for each encoding test
public class PerformanceRecord {
    String fileName;
    String method;
    long originalSize;
    long compressedSize;
    double compressionRatio;
    long encodeTime;
    long decodeTime;

    public PerformanceRecord(String fileName, String method, long originalSize, long compressedSize,
                             long encodeTime, long decodeTime) {
        this.fileName = fileName;
        this.method = method;
        this.originalSize = originalSize;
        this.compressedSize = compressedSize;
        this.compressionRatio = (double) compressedSize / originalSize;
        this.encodeTime = encodeTime;
        this.decodeTime = decodeTime;
    }

    @Override
    public String toString() {
        return String.format("%s\t%s\t%d\t%d\t%.3f\t%d\t%d",
                fileName, method, originalSize, compressedSize, compressionRatio, encodeTime, decodeTime);
    }
}
```

Figure 29. PerformanceRecord Class for Storing Encoding and Compression Metrics

- FixedLengthCoding Class:

```java
    // Encode entire text using predefined 8-bit codes
    public String encode(String text) {
        StringBuilder encoded = new StringBuilder();
        for (char c : text.toCharArray()) {
            encoded.append(str: fixedCodes.get(key: c));
        }
        return encoded.toString();
    }

    // Decode bit string back into original text
    public String decode(String encodedText, int length) {
        StringBuilder decoded = new StringBuilder();

        for (int i = 0; i < encodedText.length(); i += 8) {
            String code = encodedText.substring(beginIndex: i, i + 8);
            for (char key : fixedCodes.keySet()) {
                if (fixedCodes.get(key).equals(anObject: code)) {
                    decoded.append(c: key);
                    break;
                }
            }
        }
        return decoded.toString();
    }
```

Figure 30. Fixed-Length Encoding and Decoding Implementation

- RandomTextFileGenerator Class:

```java
import java.util.Random;
import java.io.*;

// Generates random text files with A-Z uppercase letters
public class RandomTextFileGenerator {

    public static void generateRandomFile(String fileName, int sizeKB) throws Exception {
        String characters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        Random random = new Random();
        int length = sizeKB * 1024; // Convert KB to bytes

        StringBuilder sb = new StringBuilder(capacity: length);

        // Add random characters until file reaches the desired size
        for (int i = 0; i < length; i++) {
            sb.append(c: characters.charAt(index: random.nextInt(bound: characters.length())));
        }

        FileUtils.saveFile(fileName, content: sb.toString());
    }
}
```

Figure 31. RandomTextFileGenerator Class for Creating Synthetic Test Files

- FileUtils Class:

```java
import java.io.*;
import java.nio.file.*;


// Utility class for file input/output operations
public class FileUtils {

    // Saves content into a file with given name
    public static void saveFile(String fileName, String content) throws IOException {
        Files.write(path: Paths.get(first: fileName), bytes: content.getBytes());
    }
    // Reads entire file as a String
    public static String readFile(String fileName) throws IOException {
    byte[] bytes = Files.readAllBytes(path: Paths.get(first: fileName));
    return new String(bytes);
    }

    // Returns file size in bytes
    public static long getFileSize(String fileName) throws IOException {
    return Files.size(path: Paths.get(first: fileName));
    }
}

```

Figure 32. FileUtils Class for File Input and Output Operations